Principal Component Analysis: Variance Explained & Visualizing PCA-Transformed Data

What is PCA?

Principal Component Analysis (PCA) is an unsupervised linear dimensionality reduction technique. It transforms a dataset having many features into a smaller number of new features called **Principal Components**, while preserving as much information (variance) as possible.

For Example:

Suppose We have **4 original features**.

PCA might transform these into:

- PC1
- PC2
- PC3
- PC4

If PC1 and PC2 retain 95% of the variance, we can replace the original 4 features with just **2 components**.

PCA is used to:

- Reduce the number of features while retaining most of the information (variance) in the data.
- Remove multicollinearity between correlated features.
- Visualize high-dimensional data in 2D or 3D.
- Speed up downstream machine learning algorithms and reduce overfitting/noise.

How PCA Works — Algorithm Steps

1. Standardize the data: subtract the mean and divide by the standard deviation for each feature, so that all features contribute equally (features on larger numeric scales would otherwise dominate the variance calculation).
2. Compute the covariance matrix of the standardized features, which captures how each pair of features varies together.
3. Compute eigenvalues and eigenvectors of the covariance matrix. Each eigenvector defines the direction of a principal component, and its corresponding eigenvalue represents the amount of variance captured along that direction.
4. Sort eigenvectors by descending eigenvalue, so PC1 corresponds to the largest eigenvalue (most variance), PC2 to the second largest, and so on.
5. Select the top k eigenvectors to form a projection matrix, and project the original standardized data onto this new k-dimensional subspace to obtain the PCA-transformed data.

Variance Explained

Each principal component explains a certain proportion of the total variance in the dataset. This is called the Explained Variance Ratio, calculated as:

```
Explained Variance Ratio (PCi) = eigenvalue(i) / sum(all eigenvalues)
```

The Cumulative Explained Variance is the running total of these ratios, and tells us how much of the original information is retained if we keep the first k components:

```
Cumulative Variance(k) = sum of Explained Variance Ratio for PC1 ... PCk
```

A Scree Plot displays the explained variance (bars) and cumulative explained variance (line) against each principal component. It is the standard visual tool used to decide how many components to retain — typically by looking for an 'elbow' in the curve, or by choosing enough components to cross a variance threshold such as 90% or 95%.

Choosing the Number of Components

Method	Description
Elbow Method	Pick the component after which the scree plot's individual-variance bars flatten out sharply.
Variance Threshold	Keep the smallest k such that cumulative explained variance ≥ a chosen threshold (e.g. 95%).
Kaiser's Rule	Keep components whose eigenvalue is greater than 1 (i.e., they explain more variance than a single original standardized feature).
Domain / Downstream need	Keep exactly 2 or 3 components when the sole purpose is visualization.

Visualizing PCA-Transformed Data

Once data is projected onto its principal components, it can be visualized in ways that were impossible in the original high-dimensional space:

- 2D Scatter Plot — plot PC1 vs PC2 (the two directions of maximum variance); points are typically colored by class label to visually check class separability.
- 3D Scatter Plot — plot PC1, PC2, PC3 for a richer view when 2 components do not capture enough variance.
- Biplot — overlays the 2D scatter of transformed data (scores) with arrows (loadings) showing how strongly and in which direction each original feature contributes to PC1 and PC2. Longer arrows indicate features with greater influence; the angle between arrows indicates correlation between original features.

Practical: PCA on the Iris Dataset

Dataset

The Iris dataset contains 150 samples of iris flowers from 3 species (setosa, versicolor, virginica), each described by 4 numeric features:

Feature	Description
sepal length (cm)	Length of the sepal
sepal width (cm)	Width of the sepal
petal length (cm)	Length of the petal
petal width (cm)	Width of the petal

Because the original data has 4 dimensions, it cannot be visualized directly on a single 2D scatter plot — this is exactly the kind of problem PCA is used to solve.

Procedure

1. Load the Iris dataset (150 samples × 4 features) using scikit-learn.
2. Standardize the features using StandardScaler so all 4 features have mean 0 and standard deviation 1.
3. Compute the covariance matrix and its eigenvalues/eigenvectors manually (for theoretical understanding).
4. Fit sklearn's PCA on the standardized data with all 4 components to inspect the explained_variance_ratio_ for each component.
5. Plot a Scree Plot showing individual and cumulative explained variance.
6. Reduce the data to 2 principal components and create a 2D scatter plot colored by species.
7. Create a Biplot overlaying feature loading vectors on the 2D scatter plot.
8. Reduce the data to 3 principal components and create a 3D scatter plot.
9. Automatically select the minimum number of components needed to retain ≥95% of the total variance.

Source Code

```
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D  # noqa: F401

from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

np.set_printoptions(precision=3, suppress=True)
```

```
# ------------------------------------------------------------------
# STEP 1: Load dataset
# ------------------------------------------------------------------
data = load_iris()
X = data.data                      # (150, 4) feature matrix
y = data.target                    # class labels (0,1,2)
feature_names = data.feature_names
target_names = data.target_names

print("=" * 70)
print("STEP 1: Dataset")
print("=" * 70)
print("Shape of feature matrix X:", X.shape)
print("Features:", feature_names)
print("Classes:", list(target_names))
print("\nFirst 5 rows of raw data:")
print(X[:5])

# ------------------------------------------------------------------
# STEP 2: Standardize the data (mean = 0, std = 1)
#          PCA is variance-based, so features must be on the same scale
# ------------------------------------------------------------------
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("\n" + "=" * 70)
print("STEP 2: Standardized data (mean~0, std~1)")
print("=" * 70)
print("Mean after scaling:", np.round(X_scaled.mean(axis=0), 3))
print("Std  after scaling:", np.round(X_scaled.std(axis=0), 3))

# ------------------------------------------------------------------
# STEP 3: Covariance matrix and eigen-decomposition (manual, for theory)
# ------------------------------------------------------------------
cov_matrix = np.cov(X_scaled.T)
eig_vals, eig_vecs = np.linalg.eig(cov_matrix)

# sort eigenvalues (and corresponding eigenvectors) in descending order
order = np.argsort(eig_vals)[::-1]
eig_vals = eig_vals[order]
eig_vecs = eig_vecs[:, order]

print("\n" + "=" * 70)
print("STEP 3: Covariance matrix & Eigen-decomposition")
print("=" * 70)
print("Covariance matrix:\n", cov_matrix)
print("\nEigenvalues (sorted descending):", eig_vals)
print("\nEigenvectors (columns, sorted to match eigenvalues):\n", eig_vecs)

# ------------------------------------------------------------------
# STEP 4: Fit PCA using scikit-learn (all components first, to inspect variance)
# ------------------------------------------------------------------
pca_full = PCA(n_components=X_scaled.shape[1])
X_pca_full = pca_full.fit_transform(X_scaled)

explained_var = pca_full.explained_variance_ratio_
cum_explained_var = np.cumsum(explained_var)

print("\n" + "=" * 70)
print("STEP 4: Explained Variance Ratio (sklearn PCA, all components)")
print("=" * 70)
for i, (v, c) in enumerate(zip(explained_var, cum_explained_var)):
```

```
    print(f"PC{i+1}: explained variance = {v*100:5.2f}%   cumulative = {c*100:5.2f}%")

# ---------------------------------------------------------------
# STEP 5: Scree plot (explained variance per component)
# ---------------------------------------------------------------
plt.figure(figsize=(7, 4.5))
components = np.arange(1, len(explained_var) + 1)
plt.bar(components, explained_var * 100, color="#4C72B0", alpha=0.85, label="Individual")
plt.plot(components, cum_explained_var * 100, color="#C44E52", marker="o", label="Cumulative")
plt.axhline(y=95, color="gray", linestyle="--", linewidth=1, label="95% threshold")
plt.xticks(components, [f"PC{i}" for i in components])
plt.xlabel("Principal Component")
plt.ylabel("Explained Variance (%)")
plt.title("Scree Plot - Explained Variance by Component")
plt.legend()
plt.tight_layout()
plt.savefig("/home/claude/pca_practical/scree_plot.png", dpi=150)
plt.close()
print("\nScree plot saved -> scree_plot.png")

# ---------------------------------------------------------------
# STEP 6: Reduce to 2 components and visualize
# ---------------------------------------------------------------
pca_2d = PCA(n_components=2)
X_pca_2d = pca_2d.fit_transform(X_scaled)

print("\n" + "=" * 70)
print("STEP 6: PCA reduced to 2 components")
print("=" * 70)
print("Shape after PCA:", X_pca_2d.shape)
print(f"Variance explained by PC1+PC2: {sum(pca_2d.explained_variance_ratio_)*100:.2f}%")

colors = ["#4C72B0", "#DD8452", "#55A868"]
plt.figure(figsize=(7, 5.5))
for i, name in enumerate(target_names):
    mask = y == i
    plt.scatter(X_pca_2d[mask, 0], X_pca_2d[mask, 1],
                label=name, color=colors[i], alpha=0.8, edgecolor="white", s=60)
plt.xlabel(f"PC1 ({pca_2d.explained_variance_ratio_[0]*100:.1f}% variance)")
plt.ylabel(f"PC2 ({pca_2d.explained_variance_ratio_[1]*100:.1f}% variance)")
plt.title("PCA-Transformed Iris Data (2 Components)")
plt.legend(title="Species")
plt.tight_layout()
plt.savefig("/home/claude/pca_practical/pca_2d_scatter.png", dpi=150)
plt.close()
print("2D PCA scatter plot saved -> pca_2d_scatter.png")

# ---------------------------------------------------------------
# STEP 7: Biplot (scatter + feature loading vectors)
# ---------------------------------------------------------------
loadings = pca_2d.components_.T * np.sqrt(pca_2d.explained_variance_)

plt.figure(figsize=(7.5, 6))
for i, name in enumerate(target_names):
    mask = y == i
    plt.scatter(X_pca_2d[mask, 0], X_pca_2d[mask, 1],
                label=name, color=colors[i], alpha=0.5, s=45)

scale = 3.0
for i, feature in enumerate(feature_names):
    plt.arrow(0, 0, loadings[i, 0] * scale, loadings[i, 1] * scale,
              color="black", width=0.008, head_width=0.12)
    plt.text(loadings[i, 0] * scale * 1.15, loadings[i, 1] * scale * 1.15,
```

```
            feature, color="black", fontsize=9, ha="center")

plt.axhline(0, color="gray", linewidth=0.5)
plt.axvline(0, color="gray", linewidth=0.5)
plt.xlabel(f"PC1 ({pca_2d.explained_variance_ratio_[0]*100:.1f}% variance)")
plt.ylabel(f"PC2 ({pca_2d.explained_variance_ratio_[1]*100:.1f}% variance)")
plt.title("PCA Biplot - Scores and Feature Loadings")
plt.legend(title="Species")
plt.tight_layout()
plt.savefig("/home/claude/pca_practical/pca_biplot.png", dpi=150)
plt.close()
print("Biplot saved -> pca_biplot.png")

# ---------------------------------------------------------------
# STEP 8: 3-component PCA and 3D visualization
# ---------------------------------------------------------------
pca_3d = PCA(n_components=3)
X_pca_3d = pca_3d.fit_transform(X_scaled)

fig = plt.figure(figsize=(7.5, 6))
ax = fig.add_subplot(111, projection="3d")
for i, name in enumerate(target_names):
    mask = y == i
    ax.scatter(X_pca_3d[mask, 0], X_pca_3d[mask, 1], X_pca_3d[mask, 2],
               label=name, color=colors[i], alpha=0.8, s=45)
ax.set_xlabel("PC1")
ax.set_ylabel("PC2")
ax.set_zlabel("PC3")
ax.set_title("PCA-Transformed Iris Data (3 Components)")
ax.legend(title="Species")
plt.tight_layout()
plt.savefig("/home/claude/pca_practical/pca_3d_scatter.png", dpi=150)
plt.close()
print(f"3D PCA scatter saved -> pca_3d_scatter.png "
      f"(cumulative variance PC1-3: {sum(pca_3d.explained_variance_ratio_)*100:.2f}%)")

# ---------------------------------------------------------------
# STEP 9: Choosing number of components for 95% variance retained
# ---------------------------------------------------------------
pca_95 = PCA(n_components=0.95)
X_pca_95 = pca_95.fit_transform(X_scaled)

print("\n" + "=" * 70)
print("STEP 9: Components needed to retain >= 95% variance")
print("=" * 70)
print(f"Number of components selected: {pca_95.n_components_}")
print(f"Total variance retained: {sum(pca_95.explained_variance_ratio_)*100:.2f}%")

print("\nDone.")
```

3.6 Output

Step 1 & 2 — Dataset loaded and standardized:

```
Shape of feature matrix X: (150, 4)
Features: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
Classes: ['setosa', 'versicolor', 'virginica']

Mean after scaling: [-0. -0. -0. -0.]
Std  after scaling: [1. 1. 1. 1.]
```

Step 3 — Covariance matrix and eigen-decomposition (manual computation):

```
Covariance matrix:
 [[ 1.007 -0.118  0.878  0.823]
  [-0.118  1.007 -0.431 -0.369]
  [ 0.878 -0.431  1.007  0.969]
  [ 0.823 -0.369  0.969  1.007]]

Eigenvalues (sorted descending): [2.938  0.920  0.148  0.021]
```

Step 4 — Explained variance ratio per principal component (sklearn PCA):

Component	Explained Variance	Cumulative Variance
PC1	72.96%	72.96%
PC2	22.85%	95.81%
PC3	3.67%	99.48%
PC4	0.52%	100.00%

Step 5 — Scree Plot (visualizing explained variance per component):

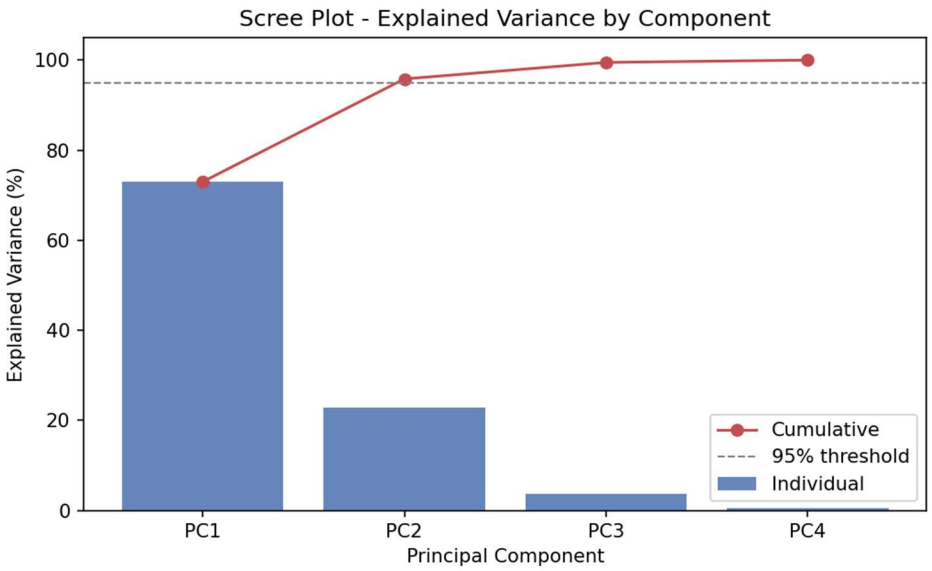

Fig 3.1 — Scree plot: PC1 alone explains ~73% of variance; PC1+PC2 together cross the 95% threshold (dashed line).

Step 6 — 2D scatter plot of PCA-transformed data (PC1 vs PC2), colored by species:

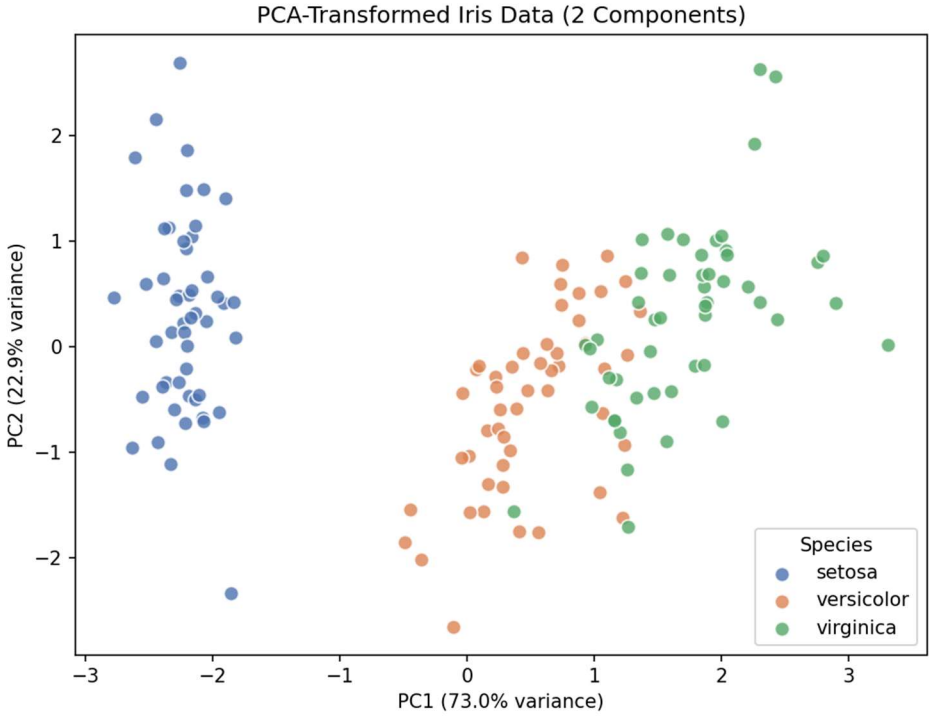

Fig 3.2 — Just 2 components (95.8% of total variance) clearly separate all three Iris species, even though the original data had 4 dimensions.

Step 7 — Biplot: PCA scores overlaid with original feature loading vectors:

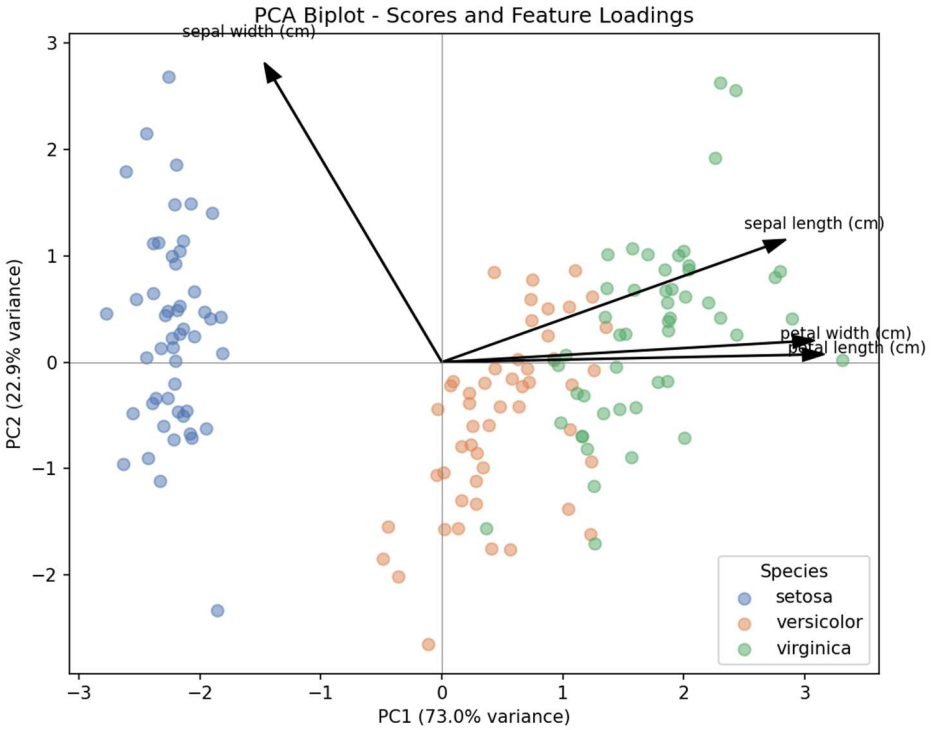

Fig 3.3 — Biplot showing that petal length and petal width point almost entirely along PC1 (they drive most of the species separation), while sepal width points mostly along PC2.

Step 8 — 3D scatter plot using the first 3 principal components (99.5% of total variance):

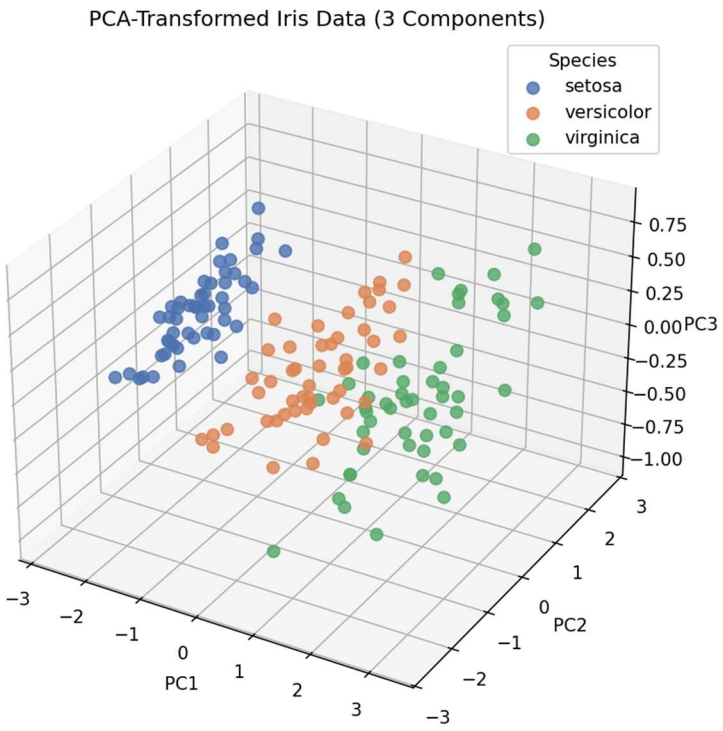

Fig 3.4 — 3D visualization gives an even clearer separation between the three species, at the cost of one extra dimension to interpret.

Step 9 — Automatic component selection for 95% variance retention:

```
Number of components selected: 2
Total variance retained: 95.81%
```